

The Shadow Price of Reasoning: Economic Perspective on Optimal Budget Allocation for LLMs

Xu Wan^{*123} Speed Zhu^{*2} Jianwei Cai² Guang Chen² Ximing Huang¹ Wiggin Zhou² Mingyang Sun¹

Abstract

Inference-time scaling has emerged as a critical avenue for enhancing Large Language Models’ performance, yet real-world deployment is constrained by strict computational budgets. In this work, we formulate inference budget allocation as a global constrained optimization problem governed by economic principles. By modeling per-query reasoning utility with a shifted-surge function, we derive an optimal allocation policy based on a global shadow price that equilibrates marginal utility under resource scarcity. Based on this theory, we propose **Constrained Latent-utility Equilibrium Allocation for Reasoning (CLEAR)**. It performs rational abandonment and reallocates resources from insolvent queries to solvable queries near their emergence thresholds. Extensive experiments on several reasoning tasks with different traffic streams demonstrate that **CLEAR** significantly improves the Pareto frontier of total token cost versus mean accuracy. In resource-scarce regimes, **CLEAR** achieves up to a $3\times$ improvement in global accuracy compared to uniform allocation.

🔗 The code is available in [Here](#).

1. Introduction

The development of Large Language Models (LLMs) is undergoing a paradigm shift from training-time scaling to inference-time scaling (Wei et al., 2022; Snell et al., 2024; Wu et al., 2024). Recent breakthroughs demonstrate that increasing test-time compute can yield performance gains comparable to scaling model parameters by invoking “System 2” reasoning capabilities (Weston & Sukhbaatar, 2023; Brown et al., 2024). A series of length-scaling experiments

¹Peking University ²Tencent HY Team ³Zhejiang University.
Correspondence to: Mingyang Sun <smy@pku.edu.cn>.

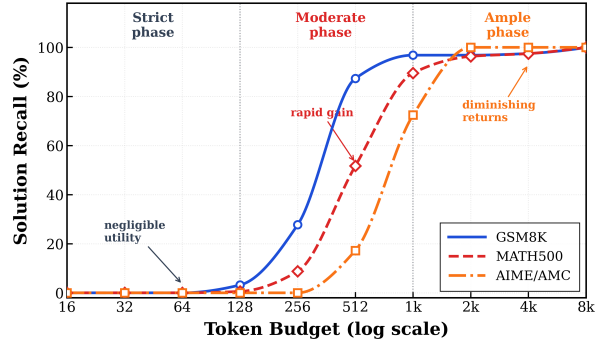


Figure 1. **The S-Shaped Compute-Utility Curve.** We evaluate Qwen2.5-Math-7B (Yang et al., 2024) on three benchmarks. The budget-performance relationship exhibits three distinct regions: (1) a pre-threshold **Strict** phase with negligible utility; (2) a rapid **Surge** phase offering high leverage; and (3) an **Ample** phase characterized by diminishing returns.

has furthermore validated that by allowing models to think longer, it leads to substantial improvements in complex reasoning tasks (Muennighoff et al., 2025; Aggarwal & Welleck, 2025). However, in real-world deployment scenarios, test-time compute is a finite and expensive commodity. Whether serving millions of concurrent users via cloud APIs or running models on resource-constrained edge devices, practitioners operate under strict global budget constraints (Chen et al., 2023). The central challenge thus shifts from merely increasing the theoretical ceiling of model intelligence to maximizing global utility under a fixed computational cap (Raposo et al., 2024).

Standard deployment practices usually impose a uniform policy, assigning the same generation limit (e.g., a fixed `max_new_tokens`) to every query. This implicitly assumes that all queries share a comparable compute-utility profile, which conflicts with the large heterogeneity of reasoning tasks. As illustrated in Figure 1, reasoning utility follows an S-shaped curve when we fix the decoding strategy and vary the token limit. For difficult queries like AIME, a uniform limit may leave the trajectory in the **Strict** phase, where compute is spent but utility remains near zero. For easy queries like GSM8K, the same limit may push generation into the **Ample** phase, where additional tokens bring only diminishing returns. Of course, in an unconstrained-

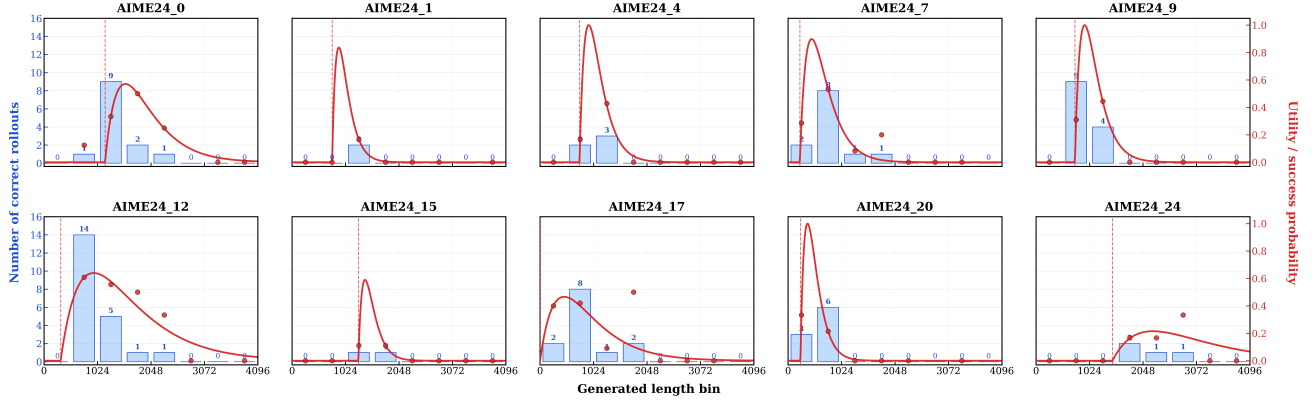


Figure 2. **Empirical Rollouts and Latent Utility.** For selected AIME-24 problems, blue bars show the number of correct rollouts in each length bin, while red curves depict the fitted latent utility mapping induced by our shifted-surge model.

resource setting, the inefficiency caused by pushing easy queries into the **Ample** phase may be less consequential. However, this work focuses on the resource-constrained setting, where a fixed total token budget is imposed over the entire evaluation set. The central question is therefore how to allocate tokens across individual queries so as to maximize overall token utility under this global budget.

We therefore treat inference budgeting as a batch-level allocation problem: given a fixed total token supply, decide how many tokens each query should receive to maximize aggregate expected utility. This gives rise to a global constrained optimization problem over instance-specific utility curves. Although the resulting objective is non-convex, its Lagrangian form reveals a simple economic principle: at optimum, every active query should spend tokens until its marginal utility matches a common *global shadow price* (Boyd & Vandenberghe, 2004; Devanur et al., 2019). Queries whose attainable marginal gain never clears this price should receive no budget, while the remaining queries share the budget according to their utility slopes.

Building on these findings, we propose **Constrained Latent-utility Equilibrium Allocation for Reasoning (CLEAR)**. Instead of a uniform utility for each query, **CLEAR** maps it into a surge-shaped utility curve. The system then operates as a computational market: (1) **Threshold Modeling**: We estimate the emergence threshold of each query and instantiate its latent utility curve; (2) **Price Discovery**: We employ a fast bisection search to find the unique global shadow price that clears the market, ensuring the total demand matches the available budget; (3) **Optimal Allocation**: Based on the discovered price, we apply a closed-form policy derived from the Lambert W function to strictly determine the token limit for each query, automatically handling truncation or rational abandonment. Crucially, **CLEAR** requires no retraining of the backbone LLM and operates as a plug-and-play inference wrapper.

Our contributions are summarized as follows:

- (1) We identify a three-phase compute-utility pattern in LLM reasoning and formalize inference-time token allocation over non-concave latent utility curves, where the optimal policy is governed by a global shadow price.
- (2) We derive a closed-form Lambert W allocation policy and instantiate it in **CLEAR**, a plug-and-play framework that combines latent-threshold prediction with market-clearing price discovery.
- (3) We validate **CLEAR** on mixed-complexity mathematical reasoning benchmarks, demonstrating improved cost-accuracy Pareto efficiency and robustness to hyperparameter choices and predictor noise.

2. Empirical Motivation

To better model the per-query utility of reasoning tokens, we investigate whether longer reasoning is uniformly beneficial for each query. If every problem has an instance-specific favorable reasoning length, then inference utility should naturally rise after a minimum threshold but eventually saturate or decline when generation becomes excessive.

Following the thoughtology analysis (Marjanović et al., 2025) in DeepSeek-R1, we conduct a controlled experiment using Qwen2.5-Math-7B. We perform sampling with a high temperature $T = 1.0$ to induce diverse reasoning paths of varying lengths. We generate $N = 50$ responses for AIME-24 and $N = 4$ for the GSM8K, MATH-500 benchmarks, and group the resulting trajectories into length bins to compute the conditional Pass@1 accuracy.

As visualized in Figure 2, reasoning utility exhibits non-linear dynamics rather than simple scaling. We identify three recurring regimes: a **Strict** phase where short trajectories fail consistently, implying a minimum solvable threshold; a **Surge** phase where performance rises sharply

past this threshold; and an **Ample** phase where additional generation yields diminishing returns and may eventually degrade solution quality.

3. Problem Formulation

Based on the empirical observations above, we first model the per-query reasoning utility with a function that captures the **Strict-Surge-Ample** structure. Under this utility model, the problem of assigning tokens subject to a fixed total budget naturally becomes a global constrained optimization problem with an instance-dependent, non-concave utility landscape.

3.1. Modeling the Physics of Reasoning

While the observed outcome of a reasoning task is binary (correct or incorrect), we posit that this outcome is governed by a continuous, unobservable variable: the reasoning utility $\phi(t)$. For a query s_i , we model the accumulated potential after generating t tokens follows a Shifted Surge Function:

$$\phi_i(t) = \begin{cases} 0 & 0 \leq t < \tau_i \\ \alpha_i(t - \tau_i) \cdot e^{-\beta_i(t - \tau_i)} & t \geq \tau_i \end{cases} \quad (1)$$

This parameterization aligns the latent utility model with the three empirical regimes observed above:

Strict: Utility remains zero until the generation length crosses the latent emergence threshold τ_i .

Surge: Once the threshold is crossed, reasoning potential rises with initial velocity α_i , capturing the rapid accumulation of valid reasoning.

Ample: When generation extends excessively, the exponential decay term $e^{-\beta_i \Delta t}$ dominates, reflecting diminishing returns.

In Figure 2, the red curve visualizes the fitted latent reasoning potential, which provides a continuous utility mapping from the empirical rollout outcomes observed at different generation lengths.

3.2. The Global Optimization Objective

We adopt a pure token-based cost model where the cost $C_i(t) = t$. The system’s goal is to allocate a vector of tokens $\mathbf{t} = [t_1, \dots, t_N]$ for N queries to maximize aggregate reasoning potential under a global token budget limit B_{total} :

$$\begin{aligned} \max_{\mathbf{t} \in \mathbb{R}_{\geq 0}^N} \quad & \sum_{i=1}^N \phi_i(t_i) \\ \text{s.t.} \quad & \sum_{i=1}^N t_i \leq B_{\text{total}}. \end{aligned} \quad (2)$$

4. Theoretical Analysis

To solve the non-convex optimization problem in Eq. (2), we apply the method of Lagrange multipliers. By relaxing the global budget constraint, we establish an economic principle governing optimal inference: the equalization of marginal reasoning potential.

4.1. Shadow Price Parity

We construct the Lagrangian $\mathcal{L}(\mathbf{t}, \lambda) = \sum_i \phi_i(t_i) - \lambda(\sum_i t_i - B_{\text{total}})$, where $\lambda \geq 0$ is the Lagrange multiplier. In economic terms, λ represents the *global shadow price* of computation—the marginal gain in total potential obtained by relaxing the budget by one token.

The Karush-Kuhn-Tucker (KKT) conditions imply that for any globally optimal allocation $\mathbf{t}^*$, the marginal gain of each active task must align with this global price. This parity determines whether a query should be lifted out of **Strict**, placed near its high-leverage **Surge**, or capped before wasting budget in **Ample**. We formalize this as:

Definition 4.1 (Shadow Price Parity). A strictly positive budget allocation $t_i^* > 0$ is a local maximum if it satisfies the first-order stationarity condition, equalizing the marginal reasoning potential to the global shadow price:

$$\frac{\partial \phi_i(t_i^*)}{\partial t_i} = \lambda. \quad (3)$$

The global optimal policy is then determined by comparing the utility surplus at this stationary point against the abandonment option ($t_i = 0$). We term this the *Rational Abandonment Condition*, where tasks whose maximum net surplus falls below zero are deemed economically insolvent and assigned zero budget.

4.2. Individual Optimal Allocation Policy

For the surge potential $\phi_i(t) = \alpha_i \Delta t e^{-\beta_i \Delta t}$, where $\Delta t = t - \tau_i$, the shadow price parity condition yields the differential equation:

$$\frac{d\phi_i}{dt} = \alpha_i e^{-\beta_i \Delta t} (1 - \beta_i \Delta t) = \lambda. \quad (4)$$

While standard algebraic methods cannot solve for t in this transcendental equation, we prove that the exact solution is given by the Lambert W function $W(z)$, defined as the inverse of $f(w) = we^w$.

Theorem 4.2 (Individual Optimal Allocation Policy). Under the shadow price parity condition, the optimal token allocation t_i^* for a given λ follows a closed-form solution:

$$t_i^*(\lambda) = \tau_i + \frac{1}{\beta_i} \left[1 - W_0 \left(\frac{\lambda e}{\alpha_i} \right) \right], \quad (5)$$

subject to the solvency constraint $\phi_i(t_i^*) > \lambda t_i^*$. Here, $W_0(\cdot)$ denotes the principal branch of the Lambert W function, and e is Euler's number.

Proof. The objective for each task i is to maximize the net economic surplus $J_i(t)$, defined as the difference between the latent reasoning potential and the opportunity cost of tokens:

$$J_i(t) = \phi_i(t) - \lambda t = \alpha_i(t - \tau_i)e^{-\beta_i(t-\tau_i)} - \lambda t. \quad (6)$$

We assume the task is in the active reasoning phase ($t \geq \tau_i$). Let $\Delta t = t - \tau_i$ denote the effective generation length. The first-order necessary condition for optimality requires the marginal potential to equal the shadow price:

$$\frac{d\phi_i}{dt} = \lambda. \quad (7)$$

Applying the product rule to differentiate the surge function with respect to t :

$$\begin{aligned} \frac{d}{dt} [\alpha_i \Delta t e^{-\beta_i \Delta t}] &= \alpha_i (1 \cdot e^{-\beta_i \Delta t} + \Delta t \cdot (-\beta_i) e^{-\beta_i \Delta t}) \\ &= \alpha_i e^{-\beta_i \Delta t} (1 - \beta_i \Delta t). \end{aligned} \quad (8)$$

Equating this to λ , we obtain the transcendental equation:

$$\alpha_i e^{-\beta_i \Delta t} (1 - \beta_i \Delta t) = \lambda. \quad (9)$$

To solve for Δt , we transform this equation into the canonical form of the Lambert W function, $we^w = z$. We introduce a change of variable:

$$u = 1 - \beta_i \Delta t. \quad (10)$$

This implies $\beta_i \Delta t = 1 - u$, and consequently, the exponential term becomes $e^{-\beta_i \Delta t} = e^{-(1-u)} = e^{u-1}$. Substituting these into the optimality condition:

$$\begin{aligned} \alpha_i \cdot e^{u-1} \cdot u &= \lambda \\ ue^u e^{-1} &= \frac{\lambda}{\alpha_i} \\ ue^u &= \frac{\lambda e}{\alpha_i}. \end{aligned} \quad (11)$$

By the definition of the Lambert W function, the solution for u is given by the principal branch W_0 , as we require the solution corresponding to the physical expansion phase:

$$u = W_0 \left(\frac{\lambda e}{\alpha_i} \right). \quad (12)$$

Finally, we substitute u back to solve for the optimal allocation t_i^* :

$$\begin{aligned} 1 - \beta_i(t_i^* - \tau_i) &= W_0 \left(\frac{\lambda e}{\alpha_i} \right) \\ \beta_i(t_i^* - \tau_i) &= 1 - W_0 \left(\frac{\lambda e}{\alpha_i} \right) \\ t_i^* &= \tau_i + \frac{1}{\beta_i} \left[1 - W_0 \left(\frac{\lambda e}{\alpha_i} \right) \right]. \end{aligned} \quad (13)$$

This interior solution is valid if and only if the resulting net surplus is positive ($J_i(t_i^*) > 0$). If the surplus is non-positive, the global optimum defaults to the boundary solution $t_i^* = 0$. $\square$

4.3. Global Equilibrium and Phase Transitions

The Lambert W policy derived in Eq. (5) describes the local optimum for a fixed price λ . However, the inference system operates under a hard global constraint. The *global optimal policy* is realized by finding the unique market-clearing price λ^* that satisfies the resource budget:

$$\sum_{i=1}^N t_i^*(\lambda^*) = B_{\text{total}}. \quad (14)$$

Since $t_i^*(\lambda)$ is strictly monotonically decreasing with respect to λ , a unique λ^* is guaranteed to exist. The magnitude of the global budget B_{total} inversely determines λ^* . This induces three system-level allocation regimes, which correspond to different operating points on the per-query **Strict-Surge-Ample** utility curves:

1. Abundant-Budget Regime. When B_{total} is large, the shadow price λ^* approaches zero. In this limit, the Lambert W term vanishes: $W_0(0) = 0$, and the allocation converges to $t_i \rightarrow \tau_i + 1/\beta_i$. Active tasks are therefore allowed to approach the peak of their utility curves, near the transition from **Surge** to **Ample**.

2. Scarce-Budget Regime. As B_{total} tightens, λ^* rises. The term $W_0(\frac{\lambda e}{\alpha_i})$ increases, compressing the allocation t_i^* toward the threshold τ_i . The system thus prioritizes queries whose allocated tokens can still land in the high-marginal-return **Surge** segment.

3. Abandonment Regime. When B_{total} is critically low, λ^* exceeds the initial velocity α_i for difficult tasks ($\lambda^* > \alpha_i$). No positive allocation can generate enough marginal surplus to justify crossing the threshold, so such tasks are abandoned and remain before their **Strict** threshold with indicator function $\mathbb{I}(\cdot) = 0$.

5. Methodology

In this section, we translate the theoretical optimal policy derived in Theorem 4.2 into **CLEAR**, a practical inference-time control algorithm. Unlike heuristic methods that rely on manual rules for truncation or abandonment, **CLEAR** numerically solves for the shadow price λ^* that equilibrates the aggregate demand of the batch with the global supply of tokens.

5.1. Threshold Prediction and Parametric Scaling

The core input to our Lambert W policy is the emergence threshold τ_i , which captures the minimum computation re-

quired for query s_i to enter the productive reasoning regime. We employ a lightweight predictor $f_\theta : \mathcal{S} \rightarrow \mathbb{R}_+$ based on DeBERTa-v3-base to estimate this value:

$$\hat{\tau}_i = \exp(f_\theta(s_i)). \quad (15)$$

To map these predictions into actionable utility curves, **CLEAR** relies on two critical parameters that govern the system’s behavior:

Initial Velocity α : This parameter defines the initial magnitude of the utility function’s derivative at the predicted threshold $\hat{\tau}$. It functions as the system’s *reservation price*: any query where the global market-clearing price λ^* exceeds α yields a negative net surplus and is assigned zero tokens.

Decay Rate β : This parameter governs the exponential rate at which marginal utility diminishes beyond the predicted length $\hat{\tau}$. It is inversely proportional to the characteristic length of the token allocation window. To accommodate varying traffic conditions, we implement an adaptive mechanism where β is derived dynamically from the aggregate budget surplus:

$$\beta = \frac{1}{\max(\epsilon, \bar{B} - \bar{\tau})}. \quad (16)$$

Here, $\bar{B}$ is the designated budget constraint and $\bar{\tau}$ is the mean predicted budget.

Remarks. While Theorem 4.2 provides a general solution for task-specific shaping parameters $\{\alpha_i, \beta_i\}$, predicting β_i of an unseen query is computationally intractable and prone to high variance. Therefore, we adopt a model-intrinsic assumption: we treat α and β as global hyperparameters that characterize the average reasoning dynamics of the LLM backbone itself, rather than the specific query. We assume task heterogeneity is primarily captured by the threshold parameter τ_i , which we predict dynamically.

5.2. Global Shadow Price Optimization

The core of **CLEAR** lies in identifying the precise λ^* that saturates the global budget constraint. Leveraging the monotonicity established in Section 4.3, where the individual optimal allocation $t_i^*(\lambda)$ strictly decreases with respect to the shadow price, we guarantee that the aggregate consumption function $C(\lambda) = \sum t_i^*(\lambda)$ is invertible. Therefore, we solve λ^* using the bisection method, proceeding in three distinct steps:

- **Search Space Initialization:** We define the search interval $[\lambda_{\min}, \lambda_{\max}]$. The lower bound $\lambda_{\min} = 0$ represents the saturation regime (infinite budget). The upper bound $\lambda_{\max} = \alpha$ corresponds to the total abandonment regime. Crucially, α serves as the theoretical

ceiling; any price $\lambda > \alpha$ yields negative surplus for all tasks, resulting in zero allocation.

- **Candidate Evaluation:** In each iteration, we propose a candidate price λ_{mid} and compute the optimal allocation for every query using the Lambert W Policy (Eq. 5). This involves three substeps: first, computing the unconstrained surge length via the principal branch W_0 using the adaptive β ; second, enforcing the solvency condition by zeroing out allocations where the cost $\lambda \cdot t$ exceeds the utility (Rational Abandonment); and third, applying the physical context limit $T_{\max}$.
- **Market Clearing Update:** We aggregate the individual allocations to determine the total consumption C_{total} . If C_{total} exceeds the global budget B_{total} , indicating excess demand, we raise the price by setting $\lambda_{\min} \leftarrow \lambda_{\text{mid}}$. Conversely, if there is a supply surplus, we lower the price by setting $\lambda_{\max} \leftarrow \lambda_{\text{mid}}$.

The full procedure is summarized in Algorithm 1. Note that the Lambert W function $W_0(z)$ is efficiently computed via standard numerical libraries, thus introducing negligible latency compared to the autoregressive generation process.

6. Experimental Settings

Models and Training Data We employ Qwen2.5-Math-7B-Instruct and Qwen3-30B-A3B-Instruct (Yang et al., 2025) as the frozen backbones for all reasoning tasks. To estimate the latent emergence threshold $\tau(s)$, we fine-tune a DeBERTa-v3-base encoder as the threshold predictor. The predictor is trained to regress the logarithmic token length of the model’s generated solutions, which serves as a proxy for the emergence threshold. The training data is drawn exclusively from splits of GSM8K and MATH.

Evaluation Datasets and Streams As illustrated in Figure 3, we evaluate on a mixed oracle pool spanning MATH-500, AMC-23, AIME-24, AIME-25, Minerva, and OlympiadBench, and construct four synthetic inference streams: Balanced, Mostly-Easy, Mostly-Hard, and U-Shaped. The detailed training and evaluation setting is shown in Table 6 and 7.

Baselines and Allocation Policies We benchmark **CLEAR** (Lambert)¹ against several allocation strategies. **Uniform** evenly distributes the global budget across all queries, while **Predictor** assigns tokens proportional to the predicted threshold $\hat{\tau}_i$ but does not support abandonment. **TALE-EP** (Han et al., 2025) serves as an external

¹We use **CLEAR** (Lambert) to denote the full **CLEAR** pipeline proposed above, distinguishing it from the internal **CLEAR** variants used for ablation.

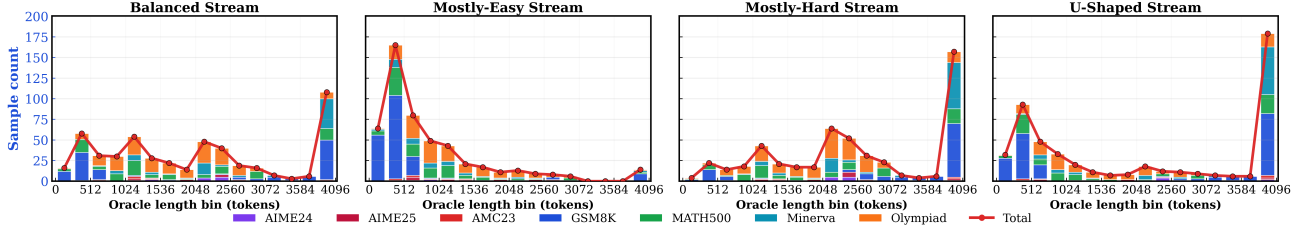


Figure 3. Oracle-Length Distribution across Evaluation Traffic Streams. Each panel shows one synthetic traffic stream (Balanced, Mostly-Easy, Mostly-Hard, and U-Shaped), with $n=500$ queries sampled from the 7B oracle pool.

length-prediction baseline that estimates per-query token demand and renormalizes allocations to the same global budget. We also compare against two internal ablations: **CLEAR** (Heuristic), which applies an affine allocation with a median-based rejection cutoff, and **CLEAR** (Auction), which greedily admits queries by predicted return on investment under the budget constraint. Finally, **Oracle** provides an upper bound using ground-truth solution lengths. Detailed formulations are provided in Appendix A.2.

7. Results and Analysis

7.1. Latent Threshold Modeling

Accurately predicting the exact token consumption for reasoning tasks is inherently challenging. However, for our resource allocation purpose, the global shadow price λ naturally acts as a normalizer, absorbing absolute prediction errors. Thus, the allocation primarily relies on the predictor’s ability to correctly rank tasks by latent threshold. Figure 4 demonstrates the log-scale regression performance of our predictor.

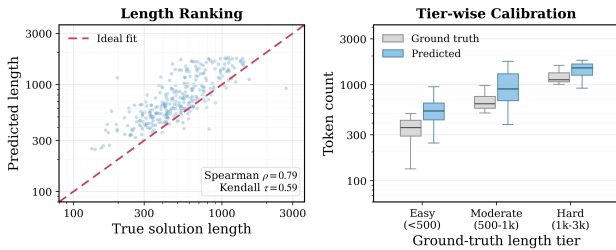


Figure 4. Predictor Performance Analysis. (Left) Log-log scatter plot of predicted vs. actual token lengths. High rank correlation coefficients (Spearman’s ρ , Kendall’s τ) indicate the model effectively captures relative threshold. (Right) Stratified comparison of token count distributions across threshold tiers.

7.2. Main Performance Comparison

Comparison on Mathematical Reasoning. Figure 6 and Table 1 summarizes the accuracy of **CLEAR** compared to the Uniform, Predictor-based, and Heuristic and Auction-based variants. The largest improvements occur under the

most constrained budget of 256 tokens per query. In the **Balanced** stream, **CLEAR** improves over the Uniform policy by **+11.6** accuracy points. In the **Mostly-Easy** stream, **CLEAR** achieves an even larger gain of **+24.0** accuracy points, indicating that uniform allocation wastes scarce tokens on queries that cannot produce sufficient utility under the global budget. In the **Mostly-Hard** and **U-Shaped** streams, **CLEAR** further improves over Uniform by **+5.2** and **+14.2** points, respectively. As the budget increases, the absolute gains naturally shrink because most methods receive enough tokens to cross the latent emergence threshold. This trend confirms that **CLEAR** is most beneficial in resource-scarce regimes, where shadow-price allocation and rational abandonment prevent the system from spreading compute too thinly across the batch.

Generalization to Code Generation To examine whether the proposed allocation principle extends beyond mathematical reasoning, we evaluate **CLEAR** on code-generation tasks. We use Qwen2.5-Coder-7B and retrain the length regressor in-domain using completion lengths from HumanEval and MBPP, while keeping the allocation algorithm unchanged. Results are reported under a best-of- K protocol with $K = 4$: each trajectory has a cap of $B = 1024$, and each query receives the same total completion-token budget of $4B$ across methods.

As shown in Table 2, **CLEAR** consistently improves over uniform allocation across all three code benchmarks. This suggests that global budget clearing and strategic abandonment are not limited to the math-focused setting, but can also benefit stochastic code generation under strict compute constraints.

Visualization of Token Allocation To understand the mechanism behind the performance of **CLEAR**, we visualize the token allocation in Figure 6.

Under the scarcity regime of 256 tokens, the **Uniform** and **Predictor**-based policies distribute the budget continuously but insufficiently across all queries. Consequently, the majority of tasks receive fewer tokens than their ground-truth reasoning length, leading to systemic failure. In con-

Table 1. Comprehensive Performance Comparison across Evaluation Streams. We compare **CLEAR** (Lambert) against various baselines including **Uniform** allocation, **Proportional Predictor**, and two **CLEAR** variants. The **Oracle** represents the theoretical upper bound. **Bold** indicates the best practical performance among non-oracle methods. The bottom row highlights the **Absolute Accuracy Gain** of **CLEAR** (Lambert) over the **Uniform**.

Method	Balanced Stream				Mostly-Easy Stream				Mostly-Hard Stream				U-Shaped Stream			
	256	512	1024	2048	256	512	1024	2048	256	512	1024	2048	256	512	1024	2048
Uniform	3.0	12.4	17.4	21.6	9.0	33.8	45.6	49.0	1.0	5.0	6.8	11.4	4.4	18.4	25.4	27.2
Predictor	0.6	4.6	18.8	22.8	1.8	17.6	46.0	49.0	0.0	1.6	8.2	12.8	0.8	9.0	26.4	27.6
TALE-EP	3.8	6.8	16.0	22.2	7.2	10.8	22.4	39.8	4.2	5.8	9.2	12.0	7.2	10.4	24.6	29.0
CLEAR (Heaur.)	11.0	17.0	18.8	22.8	31.6	37.8	46.0	49.0	4.0	7.2	8.2	12.8	17.8	20.6	26.4	27.6
CLEAR (Auction)	14.2	15.6	18.8	22.8	32.8	39.2	46.0	49.0	6.0	6.6	8.2	12.8	18.4	22.2	26.4	27.6
CLEAR (Lambert)	14.6	16.4	18.8	22.8	33.0	40.4	47.6	49.0	6.2	6.8	8.2	12.8	18.6	23.2	26.8	27.6
Oracle	19.0	21.6	23.6	26.0	42.2	48.4	49.8	49.8	9.6	11.4	12.8	16.6	26.4	27.6	30.2	32.8
Gain (Abs.)	+11.6	+4.0	+1.4	+1.2	+24.0	+6.6	+2.0	+0.0	+5.2	+1.8	+1.4	+1.4	+14.2	+4.8	+1.4	+0.4

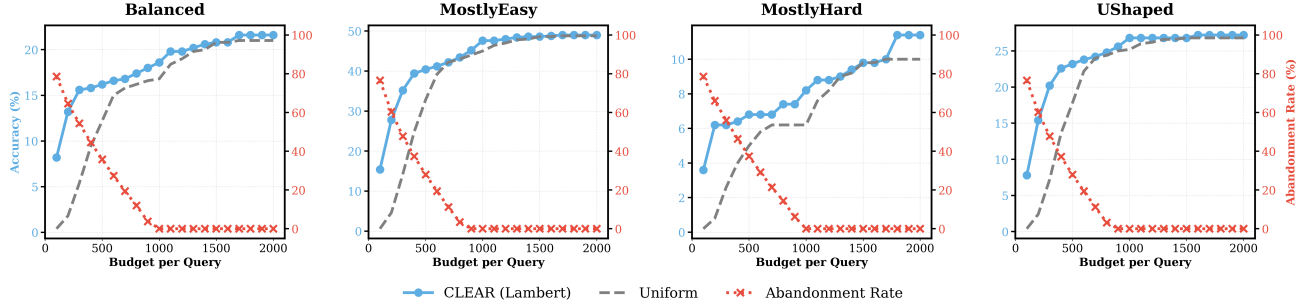


Figure 5. Phase Transition Analysis. Dual-axis plots showing Accuracy in blue and Abandonment Rate in red as the global budget increases. **CLEAR** significantly outperforms the gray dashed Uniform curve in the low-budget regime by maintaining a high abandonment rate. As the budget increases, abandonment drops to zero and the policies converge.

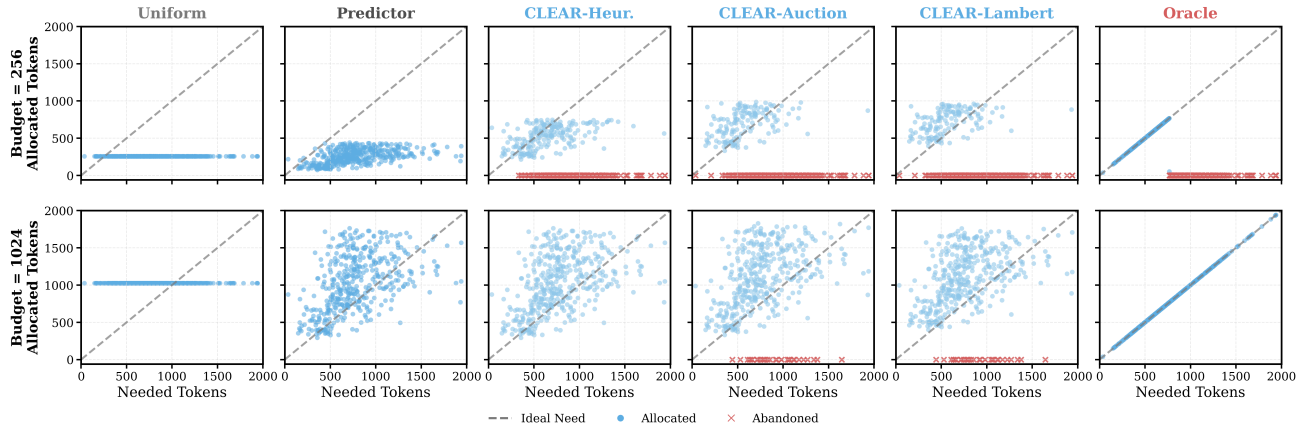


Figure 6. Visualization of Allocation Policies. Each point represents a query from the Balanced stream. The top row shows the scarcity regime with a budget of 256, while the bottom row shows the abundance regime with a budget of 1024. Blue dots represent allocated tasks, and red crosses indicate abandoned tasks.

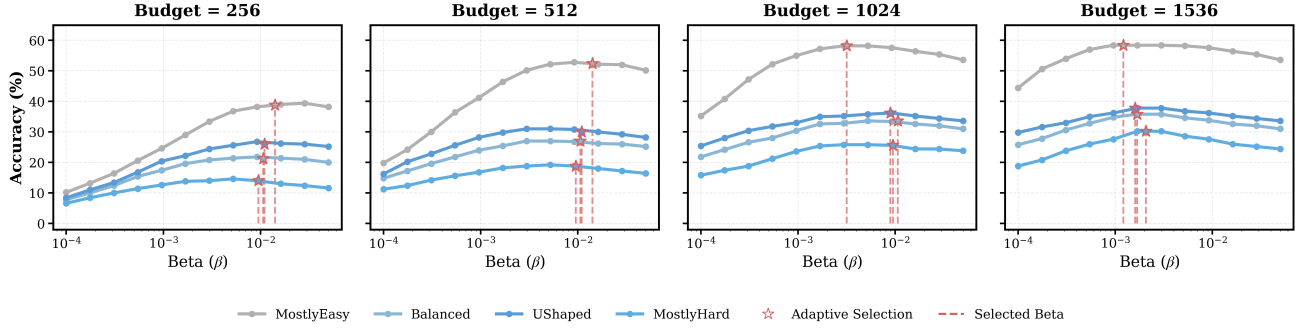


Figure 7. Sensitivity Analysis of the Decay Rate β . The star markers ($\star$) indicate the operating points automatically selected by our adaptive β mechanism.

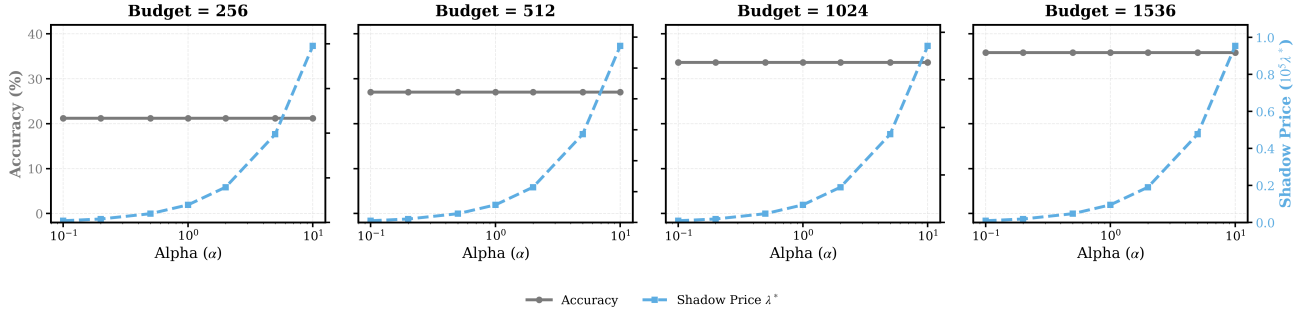


Figure 8. Scale Invariance of the Initial Velocity α . The left axis (grey) shows that accuracy remains effectively constant. The right axis (blue dashed) reveals the optimized shadow price λ^* scales linearly with α ($\log \lambda^* \propto \log \alpha$).

Table 2. Code Generation Results. Accuracy comparison under aligned best-of-4 completion-token budgets.

Method	HumanEval+	MBPP+	BigCodeBench
Uniform	36.8	39.4	16.2
CLEAR (Lambert)	43.1	45.9	19.8
Gain (Abs.)	+6.3	+6.5	+3.6

trast, **CLEAR** explicitly abandons tasks where the computational cost outweighs the potential gain. Crucially, the resources conserved from these discarded tasks are redistributed to the admitted queries. Unlike binary cut-off or auction, the Lambert policy dynamically adjusts the extra allowance based on the decay rate β , ensuring valid tasks receive ample resources to handle uncertainty.

7.3. Robustness and Sensitivity

We further analyze the robustness of **CLEAR** across three dimensions: phase transitions, predictor noise, and hyperparameter sensitivity.

Robustness to Predictor Noise. Figure 11 shows the accuracy degradation as we inject log-normal noise into the predictor. While performance naturally declines with increasing noise, **CLEAR** maintains a significant advantage

over Uniform even at high noise levels.

Sensitivity to Hyperparameters. We examine the system’s sensitivity to its two core economic parameters: the decay rate β and the initial velocity α .

Figure 7 evaluates the impact of static β values. We observe a distinct regime shift: low-budget scenarios require a strict, high β to minimize waste and ensure solvency for easier tasks, whereas high-budget scenarios benefit from a lenient, low β that allows for extended reasoning surges. Therefore, static β cannot simultaneously satisfy these conflicting demands. Our adaptive mechanism dynamically calibrates β based on the aggregate surplus, consistently landing on the Pareto-optimal frontier across all regimes without manual tuning. Unlike β , the threshold α exhibits a unique scale-invariant property. As shown in Figure 8, varying α results in negligible fluctuations in accuracy. Meanwhile, the market-clearing shadow price λ^* scales linearly with α . Since the Lambert W allocation depends on the ratio λ/α , the optimizer automatically adjusts λ^* to offset any changes in α .

7.4. Structural Utility Variants

To test whether the gains of **CLEAR** depend on the exact shifted-surge parameterization, we evaluate two alternative

utility structures that preserve the same threshold-aware allocation principle but replace the latent utility shape. The *Triangular* variant uses a tent-shaped utility that rises linearly after the predicted threshold and then decays linearly after its peak. The *Quadratic* variant uses a concave quadratic peak over the post-threshold extension. In both cases, the system still solves for a global budget-clearing allocation and applies rational abandonment under tight budgets. Let $\Delta t = t - \tau_i$ denote the effective post-threshold generation length, c_i denote the maximum supported post-threshold extension, $p_i \in (0, c_i)$ denote the triangular peak location, and $u_i^{\max}$ denote the peak utility. The *Triangular* utility is defined as:

$$\phi_i^{\text{Tri}}(t) = \begin{cases} 0 & 0 \leq t < \tau_i \\ u_i^{\max} \cdot \frac{\Delta t}{p_i} & 0 \leq \Delta t \leq p_i \\ u_i^{\max} \cdot \frac{c_i - \Delta t}{c_i - p_i} & p_i < \Delta t \leq c_i \\ 0 & \Delta t > c_i. \end{cases} \quad (17)$$

The *Quadratic* utility uses a smooth concave peak over the same post-threshold support:

$$\phi_i^{\text{Quad}}(t) = \begin{cases} 0 & 0 \leq t < \tau_i \\ u_i^{\max} \cdot \frac{4\Delta t(c_i - \Delta t)}{c_i^2} & 0 \leq \Delta t \leq c_i \\ 0 & \Delta t > c_i. \end{cases} \quad (18)$$

Table 3. **Structural Utility Variant Results.** Accuracy comparison under the tight-budget setting ($\bar{B} = 256$).

Method	Balanced	Mostly-Hard	U-Shaped
Uniform	5.4	2.2	8.2
CLEAR (Lambert)	19.4	13.4	23.8
Triangular (Lambert)	18.8	13.0	21.6
Quadratic (Lambert)	19.4	13.4	23.6

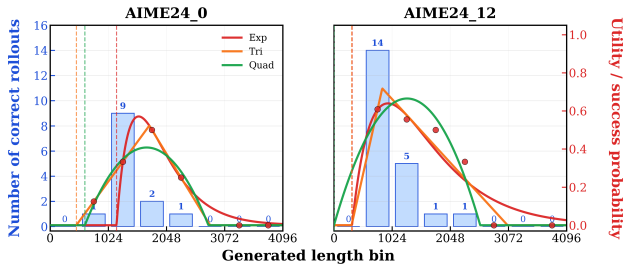


Figure 9. **Visualization of Structural Utility Variants.**

Table 3 shows that both structural variants remain substantially above uniform allocation in the most resource-constrained regime. This suggests that the main benefit comes from global budget clearing and abandonment-aware allocation, rather than from a single fragile choice of utility curve.

8. Related Work

Resource Allocation for LLM Inference. LLM deployment is increasingly studied as a resource allocation problem, where computation, latency, and tokens are scarce assets to be assigned under global constraints. Systems such as FrugalGPT route queries through model cascades to reduce cost (Chen et al., 2023), while RouteLLM learns preference-based routing policies across models (Ong et al., 2024). Other work formulates model selection or query allocation as contextual bandits, knapsack optimization, or cascade scheduling, including LLM-Bandit and TREACLE (Zhang et al., 2024). Complementary infrastructure analyses characterize the hardware-level Pareto frontier of throughput, latency, and energy (Kwon et al., 2023; Sheng et al., 2023; Wilhelm et al., 2025).

Efficient CoT. CoT prompting and inference-time scaling show that allocating more computation at test time can substantially improve reasoning performance (Wei et al., 2022; Snell et al., 2024; Wu et al., 2024; Wan et al., 2025). A related line of work improves token efficiency by adapting the reasoning procedure itself. DSC (Wang et al., 2025) allocates different numbers of sampled reasoning paths according to question difficulty and posterior confidence. Self-Budgeter (Li et al., 2025) trains the model to estimate and obey instance-specific reasoning budgets, reducing unnecessary overthinking. TALE (Han et al., 2025) estimates a token budget for each problem and injects it into the prompt to guide shorter CoT generation. These methods focus on controlling or aggregating reasoning traces within a query. In contrast, **CLEAR** addresses a batch-level allocation problem: given fixed decoding behavior and a fixed global token budget, it decides how much budget each query should receive and when a query should be rationally abandoned.

9. Conclusion

In this work, we study how to allocate a fixed inference-time token budget across heterogeneous reasoning queries. We show that uniform budgeting can waste tokens on hard queries that cannot be solved within the available budget, while overlooking queries that could be solved with a more targeted allocation. To address this mismatch, we formulate batch-level token allocation as a constrained optimization problem and introduce **CLEAR**. **CLEAR** assigns zero budget to queries with negative expected surplus and reallocates tokens to queries where additional computation is more likely to improve utility. Extensive experiments show that **CLEAR** improves the cost-accuracy Pareto frontier, with the largest gains appearing in resource-scarce regimes where uniform allocation spreads computation too thinly. Future work will focus on narrowing the gap toward oracle-level allocation by improving the robustness of utility modelling.

Impact Statement

This paper presents work whose goal is to advance the field of LLM and machine learning. There are many potential societal consequences of our work, none which we feel must be specifically highlighted here.

Acknowledgements

This work was supported by the National Natural Science Foundation of China under Grant 72571007 and Grant 72595830/72595831.

References

- Aggarwal, P. and Welleck, S. L1: Controlling how long a reasoning model thinks with reinforcement learning. *arXiv preprint arXiv:2503.04697*, 2025.
- Boyd, S. and Vandenberghe, L. *Convex optimization*. Cambridge university press, 2004.
- Brown, B., Juravsky, J., Ehrlich, R., Clark, R., Le, Q. V., Ré, C., and Mirhoseini, A. Large language monkeys: Scaling inference compute with repeated sampling. *arXiv preprint arXiv:2407.21787*, 2024.
- Chen, L., Zaharia, M., and Zou, J. Frugalgpt: How to use large language models while reducing cost and improving performance. *arXiv preprint arXiv:2305.05176*, 2023.
- Devanur, N. R., Jain, K., Sivan, B., and Wilkens, C. A. Near optimal online algorithms and fast approximation algorithms for resource allocation problems. *Journal of the ACM (JACM)*, 66(1):1–41, 2019.
- Han, T., Wang, Z., Fang, C., Zhao, S., Ma, S., and Chen, Z. Token-budget-aware llm reasoning. In *Findings of the Association for Computational Linguistics: ACL 2025*, pp. 24842–24855, 2025.
- Kwon, W., Li, Z., Zhuang, S., Sheng, Y., Zheng, L., Yu, C. H., Gonzalez, J., Zhang, H., and Stoica, I. Efficient memory management for large language model serving with pagedattention. In *Proceedings of the 29th symposium on operating systems principles*, pp. 611–626, 2023.
- Li, Z., Dong, Q., Ma, J., Zhang, D., Jia, K., and Sui, Z. Selfbudgeter: Adaptive token allocation for efficient llm reasoning. *arXiv preprint arXiv:2505.11274*, 2025.
- Marjanović, S. V., Patel, A., Adlakha, V., Aghajohari, M., BehnamGhader, P., Bhatia, M., Khandelwal, A., Kraft, A., Krojer, B., Lù, X. H., et al. Deepseek-r1 thoughtology: Let’s think about llm reasoning. *arXiv preprint arXiv:2504.07128*, 2025.
- Muennighoff, N., Yang, Z., Shi, W., Li, X. L., Fei-Fei, L., Hajishirzi, H., Zettlemoyer, L., Liang, P., Candès, E., and Hashimoto, T. B. s1: Simple test-time scaling. In *Proceedings of the 2025 Conference on Empirical Methods in Natural Language Processing*, pp. 20286–20332, 2025.
- Ong, I., Almahairi, A., Wu, V., Chiang, W.-L., Wu, T., Gonzalez, J. E., Kadous, M. W., and Stoica, I. Routellm: Learning to route llms with preference data. *arXiv preprint arXiv:2406.18665*, 2024.
- Raposo, D., Ritter, S., Richards, B., Lillicrap, T., Humphreys, P. C., and Santoro, A. Mixture-of-depths: Dynamically allocating compute in transformer-based language models. *arXiv preprint arXiv:2404.02258*, 2024.
- Sheng, Y., Zheng, L., Yuan, B., Li, Z., Ryabinin, M., Chen, B., Liang, P., Ré, C., Stoica, I., and Zhang, C. Flexgen: High-throughput generative inference of large language models with a single gpu. In *International Conference on Machine Learning*, pp. 31094–31116. PMLR, 2023.
- Snell, C., Lee, J., Xu, K., and Kumar, A. Scaling llm test-time compute optimally can be more effective than scaling model parameters. *arXiv preprint arXiv:2408.03314*, 2024.
- Wan, X., Wang, W., Xu, W., Yin, W., Song, J., and Sun, M. Adapthink: Adaptive thinking preferences for reasoning language model. *arXiv preprint arXiv:2506.18237*, 2025.
- Wang, X., Feng, S., Li, Y., Yuan, P., Zhang, Y., Tan, C., Pan, B., Hu, Y., and Li, K. Make every penny count: Difficulty-adaptive self-consistency for cost-efficient reasoning. In *Findings of the Association for Computational Linguistics: NAACL 2025*, pp. 6904–6917, 2025.
- Wei, J., Wang, X., Schuurmans, D., Bosma, M., Xia, F., Chi, E., Le, Q. V., Zhou, D., et al. Chain-of-thought prompting elicits reasoning in large language models. *Advances in neural information processing systems*, 35:24824–24837, 2022.
- Weston, J. and Sukhbaatar, S. System 2 attention (is something you might need too). *arXiv preprint arXiv:2311.11829*, 2023.
- Wilhelm, P., Wittkopp, T., and Kao, O. Beyond test-time compute strategies: Advocating energy-per-token in llm inference. In *Proceedings of the 5th Workshop on Machine Learning and Systems*, pp. 208–215, 2025.
- Wu, Y., Sun, Z., Li, S., Welleck, S., and Yang, Y. Inference scaling laws: An empirical analysis of compute-optimal inference for problem-solving with language models. *arXiv preprint arXiv:2408.00724*, 2024.

Yang, A., Zhang, B., Hui, B., Gao, B., Yu, B., Li, C., Liu, D., Tu, J., Zhou, J., Lin, J., et al. Qwen2. 5-math technical report: Toward mathematical expert model via self-improvement. *arXiv preprint arXiv:2409.12122*, 2024.

Yang, A., Li, A., Yang, B., Zhang, B., Hui, B., Zheng, B., Yu, B., Gao, C., Huang, C., Lv, C., et al. Qwen3 technical report. *arXiv preprint arXiv:2505.09388*, 2025.

Zhang, X., Huang, Z., Taga, E. O., Joe-Wong, C., Oymak, S., and Chen, J. Efficient contextual llm cascades through budget-constrained policy learning. *Advances in Neural Information Processing Systems*, 37:91691–91722, 2024.

A. Appendix: Algorithm Details

A.1. Pseudocode

We present the pseudocode for the **CLEAR** in Algorithm ??, which computes the optimal resource allocation vector under a given total budget constraint.

A.2. Allocation Policy Details

In this section, we formally define the token allocation t_i for a batch of queries $\mathcal{S} = \{s_1, \dots, s_N\}$ under a total budget constraint B_{total} . Let $\hat{\tau}_i$ denote the predicted emergence threshold used by practical policies, and let d_i denote the ground-truth solution length used only by the **Oracle** policy. The average budget per query is $\bar{B} = B_{\text{total}}/N$.

A.2.1. UNIFORM POLICY

This is the standard baseline where resources are distributed equally, agnostic to task complexity.

$$t_i^{\text{unif}} = \lfloor \bar{B} \rfloor. \quad (19)$$

A.2.2. PREDICTOR-BASED PROPORTIONAL POLICY

This policy assumes a linear relationship between the latent threshold and required tokens, scaling the budget proportionally to the predicted length without abandonment.

$$t_i^{\text{prop}} = \frac{\hat{\tau}_i}{\sum_{j=1}^N \hat{\tau}_j} \cdot B_{\text{total}}. \quad (20)$$

In practice, we clip the allocation to a minimum (e.g., 32 tokens) and maximum context length.

A.2.3. TALE-EP POLICY

TALE-EP first uses a stronger teacher model to estimate per-query completion needs, then enforces both a soft prompt-level budget constraint and a hard decoding cap during regeneration. Let $\tilde{\tau}_i$ denote the raw budget estimate from the teacher model (e.g., Qwen3-30B-Instruct).

1. **Teacher-based budget estimation:** For each query s_i , the teacher predicts a raw token need $\tilde{\tau}_i$.
2. **Renormalized proportional allocation:** The raw estimates are converted into per-query caps under the same total budget:

$$\hat{t}_i^{\text{tale}} = \frac{\tilde{\tau}_i}{\sum_{j=1}^N \tilde{\tau}_j} \cdot B_{\text{total}}. \quad (21)$$

In practice, we apply clipping and integerization:

$$t_i^{\text{tale}} = \text{clip}(\lfloor \hat{t}_i^{\text{tale}} \rfloor, t_{\min}, t_{\max}), \quad (22)$$

followed by a residual correction step to ensure exact budget conservation:

$$\sum_{i=1}^N t_i^{\text{tale}} = B_{\text{total}}. \quad (23)$$

3. **Budget-conditioned regeneration:** For each query s_i , we construct a budget-aware prompt with a soft instruction (e.g., “keep the entire response within at most t_i^{tale} completion tokens”), and run decoding with a hard cap:

$$y_i \sim p_{\theta}(\cdot \mid s_i, t_i^{\text{tale}}), \quad \text{max_tokens} = t_i^{\text{tale}}. \quad (24)$$

We use greedy decoding in our setup (temperature = 0, top_p = 1).

Final accuracy is computed by applying `math_verify` to regenerated outputs $\{y_i\}_{i=1}^N$.

A.2.4. CLEAR(HEURISTIC) POLICY

This baseline uses a fixed cutoff rule based on the predicted thresholds. When the average budget is too small relative to the average predicted threshold, it drops the harder half of the batch and reallocates the budget to the remaining queries.

1. **Check budget scarcity:** Let $\hat{\mathcal{T}} = \{\hat{\tau}_1, \dots, \hat{\tau}_N\}$ and define $\eta = \bar{B}/\mathbb{E}[\hat{\mathcal{T}}]$. If $\eta < 0.8$, the cutoff rule is activated.
2. **Select easier queries:** Queries with $\hat{\tau}_i > \text{Median}(\hat{\mathcal{T}})$ receive zero budget. The remaining queries share the total budget using:

$$t_i^{\text{heur}} = \begin{cases} \mu_{\text{sur}} + \kappa \cdot (\hat{\tau}_i - \mu_{\text{sur}}) & \text{if } \hat{\tau}_i \leq \text{Median}(\hat{\mathcal{T}}) \\ 0 & \text{otherwise,} \end{cases} \quad (25)$$

where μ_{sur} is the mean predicted threshold among selected queries, and κ is chosen so that the selected queries use exactly B_{total} tokens in total.

A.2.5. CLEAR(AUCTION) POLICY

This ablation uses a simple greedy selection rule. It first keeps the queries with smaller predicted thresholds, since they are more likely to be completed under a limited budget, and then distributes the available tokens among the selected queries.

1. **Sort by predicted length:** Sort the query indices by increasing predicted threshold, yielding a permutation $(p_1, \dots, p_N)$ such that $\hat{\tau}_{p_1} \leq \dots \leq \hat{\tau}_{p_N}$.
2. **Select survivors:** Keep the largest prefix of this sorted list whose predicted thresholds fit within the total budget:

$$m^* = \max \left\{ m \in [1, N] \mid \sum_{j=1}^m \hat{\tau}_{p_j} \leq B_{\text{total}} \right\}. \quad (26)$$

3. **Allocate to survivors:** Assign zero budget to all non-selected queries. The selected queries then share the full budget using the same affine allocation rule as CLEAR (Heuristic), rescaled so that $\sum_i t_i = B_{\text{total}}$.

A.2.6. ORACLE POLICY

This policy is an upper-bound baseline that uses ground-truth solution lengths d_i , which are unavailable at test time. It spends tokens on the shortest solvable queries first, maximizing the number of completed tasks under the total budget.

1. **Sort by true length:** Sort indices into a permutation $(o_1, \dots, o_N)$ such that $d_{o_1} \leq \dots \leq d_{o_N}$.
2. **Fill the budget greedily:** Allocate exactly d_{o_j} tokens to each query in sorted order until the next query would exceed B_{total} :

$$t_{o_j}^{\text{oracle}} = \begin{cases} d_{o_j} & \text{if } \sum_{l=1}^j d_{o_l} \leq B_{\text{total}} \\ 0 & \text{otherwise.} \end{cases} \quad (27)$$

B. Appendix: More Results

To demonstrate the generalizability of our framework to larger-scale reasoning models, we present additional experimental results using Qwen3-30B-A3B-Instruct.

Figure 10 validates the threshold predictor’s accuracy on this larger model, while Table 4 details the downstream allocation performance across varying supply-demand scenarios. In summary, CLEAR consistently outperforms the uniform baseline across all tested conditions, achieving up to **+2.4** accuracy gains in strictly constrained budget environments where efficient resource distribution is most critical.

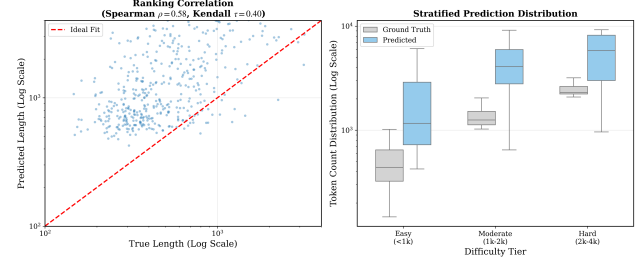


Figure 10. Predictor Performance Analysis on Qwen3-30B-A3B-Instruct.

C. Appendix: Data Composition and Statistics

Training and Test Data Table 6 and Table 7 provide a detailed breakdown of the datasets used for training the threshold predictor and evaluating the budget allocation policies. To ensure the robustness of our CLEAR framework, the evaluation suite is intentionally designed to be Out-of-Distribution (OOD) relative to the predictor’s training set.

Table 4. **Performance Comparison across Difficulty Scenarios and Budgets on Qwen3-30B-A3B-Instruct.** We compare our proposed **CLEAR** against Uniform, Direct Predictor, and heuristic CLEAR variants. Results are reported as accuracy (%). **Bold** indicates the best performance among non-oracle methods. The bottom row shows the absolute accuracy gain of CLEAR over the Uniform baseline.

Method	Balanced Stream		Mostly-Easy		Mostly-Hard		U-Shaped	
	1024	4096	1024	4096	1024	4096	1024	4096
Uniform	16.4	26.4	46.6	54.4	6.4	14.2	26.6	30.0
Predictor	8.0	28.0	32.4	55.6	1.8	15.2	13.4	31.6
CLEAR (Heur.)	18.4	28.0	37.2	55.6	9.8	15.2	23.4	31.6
CLEAR (Auction)	16.2	24.8	41.0	54.2	7.0	13.0	24.4	29.6
CLEAR (Ours)	18.8	26.8	47.4	54.6	8.6	14.8	27.2	30.6
Oracle	28.8	38.0	55.4	57.8	17.4	27.8	33.2	38.0
<i>Gain (vs. Uniform)</i>	+2.4	+0.4	+0.8	+0.2	+2.2	+0.6	+0.6	+0.6

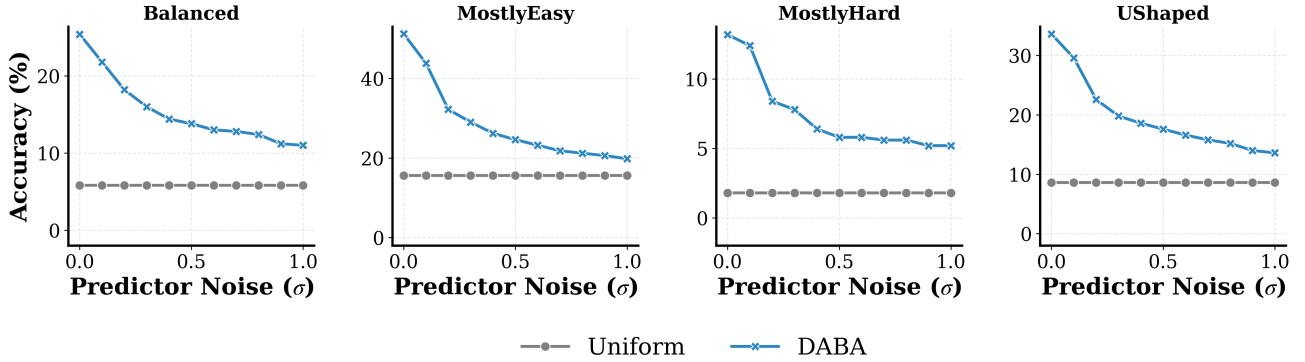


Figure 11. **Robustness to Predictor Noise.** Performance of CLEAR versus Uniform under increasing predictor noise σ . CLEAR maintains a performance advantage over the baseline even under significant noise levels.

Table 5. Detailed Experimental Configuration. This table summarizes the hyperparameters and settings used for data generation, predictor training, and the CLEAR allocation mechanism across all experiments.

Component	Parameter / Setting	Value / Description
1. Data Generation (Oracle)		
	Backbone Models	Qwen2.5-Math-7B, Qwen3-30B-A3B-Instruct
	Max New Tokens	16,384 (for 30B), 8,192 (for 7B)
	Decoding Strategy	Greedy Decoding (Temperature = 0)
2. Threshold Predictor		
	Architecture	DeBERTa-v3-base (86M parameters)
	Training Sources	GSM8K (Train), MATH (Train)
	Input Tokenization	Left-Truncation (Retain last 512 tokens)
	Max Sequence Length	512 tokens
	Training Objective	Mean Squared Error (MSE) on Log-Length
	Optimization	AdamW (LR=2e-5, Weight Decay=0.01, Batch=32)
	Training Schedule	10 Epochs
3. CLEAR Allocation Mechanism		
	Global Parameters	$\alpha = 2.0$
	Optimization Method	Bisection Search (40 iterations, $\epsilon = 1e - 6$)
4. Evaluation Scenarios		
	Sample Size	$N = 5,00$ queries per simulation stream
	Evaluation Sources	MATH-500, OlympiadBench, AIME (24/25), AMC-23, Minerva

Table 6. Detailed Statistics of Training and Evaluation Datasets for Qwen-2.5-math-7B-Instruct under greedy decoding and **4K** new token constraints.

Use Case	Dataset	Split	Total	Correct	Pass@1	Avg. Len	Threshold Tier
<i>Predictor Training</i>	GSM8K	Train	1,319	1,025	78.5%	77.71	Easy
	MATH	Train	12,500	9,477	75.8%	658	Moderate
<i>Performance Eval.</i>	MATH 500	Test	500	354	70.8%	1,074	Moderate
	OlympiadBench	Test	675	88	13.0%	1,059	Hard
	AMC-23	Test	40	19	47.5%	1,490	Hard
	AIME 2024	Test	30	8	26.7%	1,369	Very Hard
	AIME 2025	Test	30	2	6.7%	1,886	Very Hard
	Minerva	Test	272	35	12.9%	2,274	Very Hard

Table 7. Detailed Statistics of Training and Evaluation Datasets for Qwen3-30B-A3B-Instruct under greedy decoding and **16K** new token constraints. The threshold tiers are assigned based on the average reasoning length of these solutions.

Use Case	Dataset	Split	Total	Correct	Pass@1	Avg. Len	Threshold Tier
<i>Predictor Training</i>	GSM8K	Train	1,055	935	88.6%	1,254	Easy
	MATH	Train	12,500	8,246	66.0%	3,763	Moderate
<i>Performance Eval.</i>	MATH 500	Test	500	288	57.6%	9,887	Moderate
	OlympiadBench	Test	675	179	26.5%	9,450	Hard
	AMC-23	Test	40	14	35.0%	13,958	Hard
	AIME 2024	Test	30	3	10.0%	14,428	Very Hard
	AIME 2025	Test	30	14	46.7%	14,425	Very Hard
	Minerva	Test	272	37	13.6%	10,771	Very Hard

Algorithm 1 **CLEAR** (Lambert)

```

1: Input: Batch  $\mathcal{S} = \{s_1, \dots, s_N\}$ , total budget  $B_{\text{total}}$ ,
   token cap  $T_{\text{max}}$ 
2: Params: Initial velocity  $\alpha$ , tolerance  $\epsilon$ 
3: Output: Allocation vector  $\mathbf{t}^*$ 
4:  $\hat{\tau} \leftarrow \text{PredictThreshold}(\mathcal{S})$ 
5:  $\bar{B} \leftarrow B_{\text{total}}/N$ ,  $\bar{\tau} \leftarrow \frac{1}{N} \sum_{i=1}^N \hat{\tau}_i$ 
6:  $\beta \leftarrow 1/\max(\epsilon, \bar{B} - \bar{\tau})$  {batch-adaptive decay rate}
7:  $\mathbf{t}^{\text{best}} \leftarrow \mathbf{0}$ 
8: // Bisection search for the shadow price  $\lambda^*$ 
9:  $\lambda_{\min} \leftarrow 0$ ,  $\lambda_{\max} \leftarrow \alpha - 10^{-6}$ 
10: while  $(\lambda_{\max} - \lambda_{\min}) > \epsilon$  do
11:    $\lambda \leftarrow (\lambda_{\min} + \lambda_{\max})/2$ 
12:    $C_{\text{current}} \leftarrow 0$ 
13:   for  $i = 1$  to  $N$  do
14:     if  $\lambda \geq \alpha$  then
15:        $t_i \leftarrow 0$ 
16:     else
17:        $z \leftarrow (\lambda e)/\alpha$ 
18:        $u \leftarrow W_0(z)$ 
19:        $\Delta t_i \leftarrow (1 - u)/\beta$ 
20:       if  $\Delta t_i \leq 0$  then
21:          $t_i \leftarrow 0$ 
22:       else
23:          $t_i^{\text{unc}} \leftarrow \hat{\tau}_i + \Delta t_i$ 
24:          $\phi_i \leftarrow \alpha \Delta t_i \exp(-\beta \Delta t_i)$ 
25:         if  $\phi_i > \lambda t_i^{\text{unc}}$  then
26:            $t_i \leftarrow \min(t_i^{\text{unc}}, T_{\text{max}})$ 
27:         else
28:            $t_i \leftarrow 0$ 
29:         end if
30:       end if
31:     end if
32:      $C_{\text{current}} \leftarrow C_{\text{current}} + t_i$ 
33:   end for
34:   if  $C_{\text{current}} > B_{\text{total}}$  then
35:      $\lambda_{\min} \leftarrow \lambda$ 
36:   else
37:      $\lambda_{\max} \leftarrow \lambda$ 
38:      $\mathbf{t}^{\text{best}} \leftarrow \mathbf{t}$ 
39:   end if
40: end while
41:  $\mathbf{t}^* \leftarrow \lfloor \mathbf{t}^{\text{best}} \rfloor$ 
42: Clip each  $t_i^*$  to  $[0, T_{\text{max}}]$  and optionally correct residual
   tokens.
43: return  $\mathbf{t}^*$ 

```
